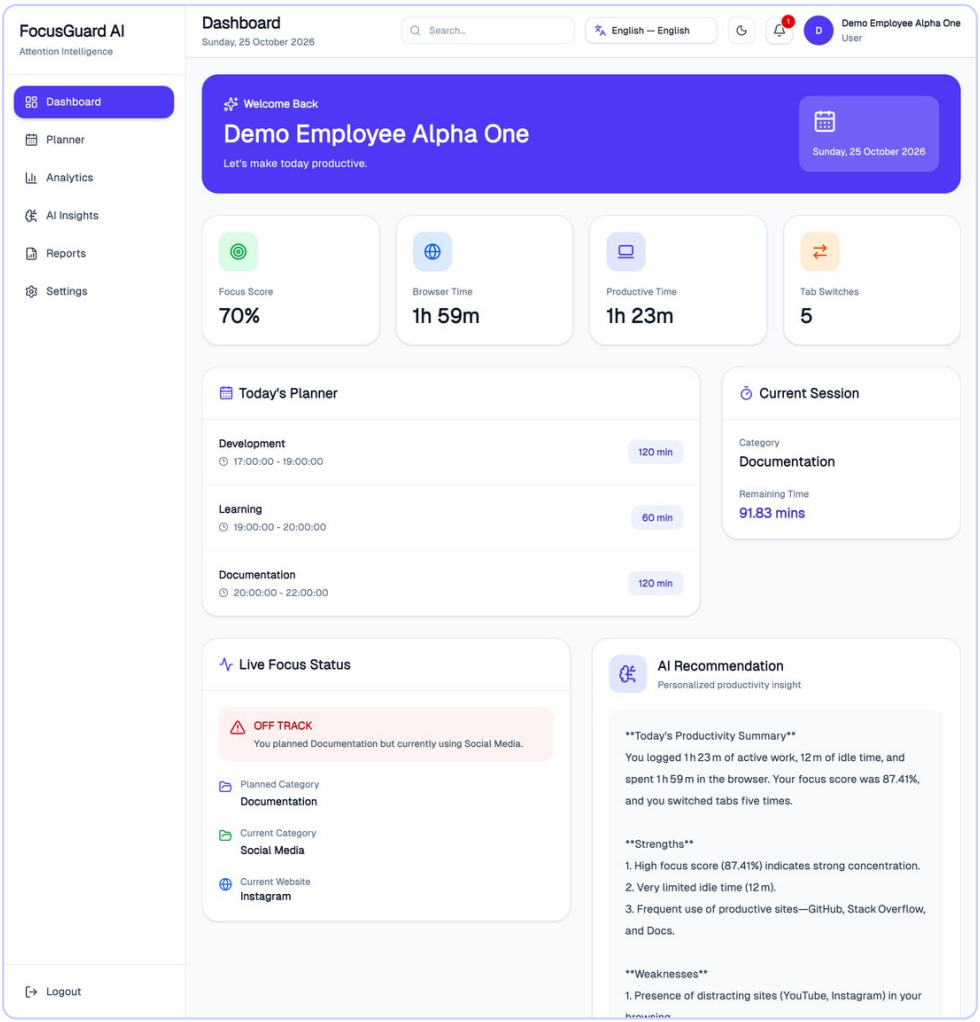
FocusGuard AI

Workflow and system overview

How the Chrome extension, the FastAPI backend, PostgreSQL with pgvector and the LLM layer work together, from a browser tab to a focus score, a recommendation and a nightly memory.

Version 1.0.0 | 31 October 2026 | github.com/buildwithabhinav-ai/focusguard-ai



Employee dashboard: focus score, planner and live focus status.

1. What FocusGuard AI does

Most screen-time tools report how long you were online. FocusGuard AI compares what you planned to work on with what you actually did, scores the difference, and explains it in plain language.

A Chrome extension records which sites you visit, for how long, when you switch tabs and when you go idle. A FastAPI backend turns those records into analytics. An LLM turns the analytics into advice. Each night the previous day is summarized and stored as an embedding in PostgreSQL, so later answers can refer to earlier weeks and not only to today.

It works for one person or for a whole organization. Every role sees only the data it is allowed to see.

The parts

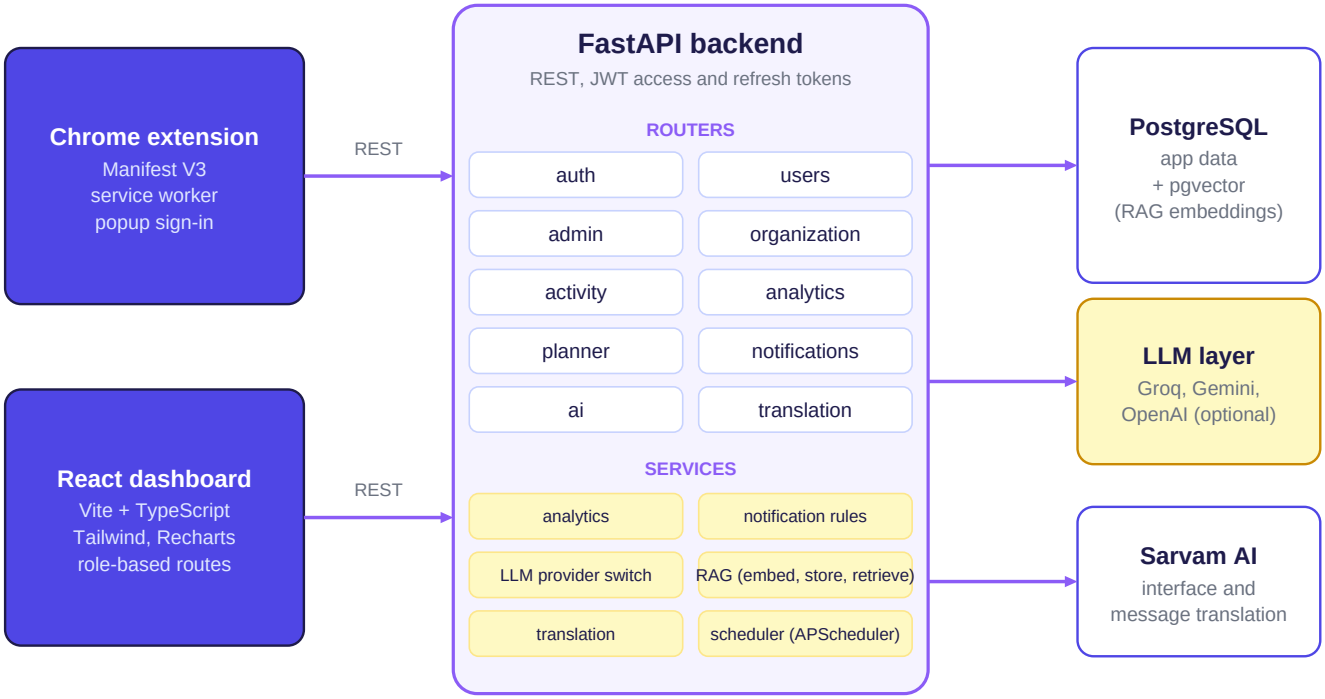
Part	Built with	Job
Chrome extension	Manifest V3, JavaScript	Records activity, tab switches and idle status. Shows Chrome notifications. Checks planner focus.
Web dashboard	React 19, TypeScript, Vite, Tailwind CSS	Role-specific dashboards, analytics, planner, AI insights, chat assistant, reports, settings.
Backend API	Python, FastAPI, SQLAlchemy, Uvicorn	Authentication, roles, analytics, planner, notifications, AI endpoints, translation.
Database	PostgreSQL with pgvector	Users, organizations, activity, plans, notifications, translations and RAG embeddings.
AI layer	Groq, Gemini, OpenAI (optional)	Writes recommendations and chat replies. Tries the preferred provider, then the others.
Embeddings	all-MiniLM-L6-v2	Turns daily summaries into 384-dimension vectors for similarity search.
Translation	Sarvam AI	Translates interface text and dynamic messages into the user's language.

Roles

Role	What they can do
Super Admin	Creates organizations, invites Sub Admins, approves or rejects deactivation requests, re-activates organizations, and sees every organization's users and analytics. The first account registered in a fresh database becomes the Super Admin.
Sub Admin	Works inside one organization: adds users, deactivates users, requests deactivation of the organization, and sees team analytics, live status and reports for that organization only.
User	Sees their own dashboard, analytics, AI insights, planner, notifications and reports, and talks to the chat assistant about their own data. Public sign-up always creates this role.

2. System architecture

Two clients talk to one backend over REST. The backend owns all data and every call to an outside AI service, so API keys never reach the browser.



What each router does

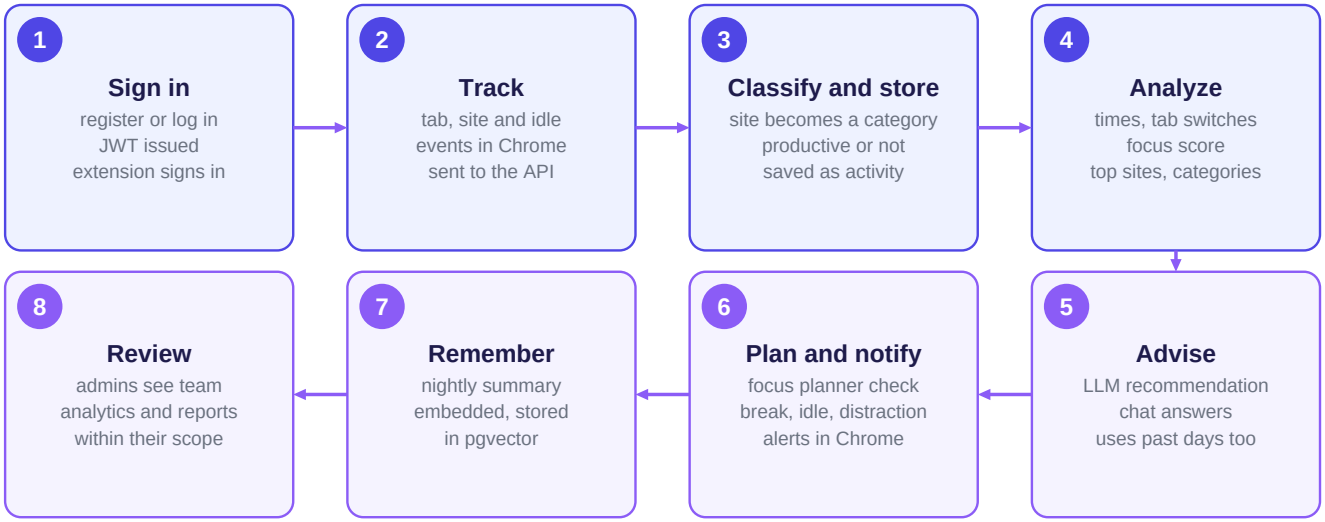
Router	Endpoints cover
auth, users	Register, login, token refresh, change password, logout, the signed-in user's profile.
admin	Super Admin dashboard, all users, change role, delete user, statistics, invite and accept Sub Admin.
organization	Create organizations, assign users, deactivation requests, approve, reject, activate, create and deactivate users.
activity	Start and end activity, current and past activity, tab switches, ACTIVE or IDLE status.
analytics	Summary, categories, websites, tab switches, dashboard and reports, for self, a role or one user.
planner	Create and update plans, today's plan, progress, current session, live status, focus check.
notifications	List, unread count, latest, mark one or all as read.
ai	User and organization recommendations, chat assistant.
translation	Fetch, create and update translations.

How it is wired

- Authentication uses JWT access tokens with refresh tokens. The extension signs in through its own popup and stores the tokens in `chrome.storage`.
- On startup the backend creates missing tables, seeds the default website categories and starts the nightly RAG scheduler.
- `DEFAULT_LLM` picks the first LLM provider. With `groq`, the order is Groq, Gemini, OpenAI. A provider without a key is skipped.

3. End-to-end workflow

One cycle runs from the first sign-in to the nightly memory, then repeats every day. Steps 2 to 6 also run continuously while the browser is open.



Step	What happens	Where it lives
1. Sign in	A person registers or logs in on the web app. The extension has its own login popup and keeps the tokens it receives.	auth router, popup.js, storage.js
2. Track	The extension watches tab, window and idle events and sends them to the API.	background.js, activity router
3. Classify and store	When an activity starts, the backend looks the URL up in the website categories table, then saves a row with category and productivity (Productive or Non Productive).	activity router, website_categories
4. Analyze	Active, idle, browser, productive and non-productive time, tab switches, top websites and categories, plus the focus score.	analytics_service.py
5. Advise	The LLM turns the numbers into a recommendation for today, the week or the month, and answers chat questions.	ai router, llm.py, prompts.py
6. Plan and notify	The planner compares live activity to the planned session. Rules create break, idle and distraction notifications.	planner router, notification_rules.py
7. Remember	Every night the previous day is summarized, embedded and stored for later retrieval.	services/rag, scheduler.py
8. Review	Sub Admins and the Super Admin open analytics and reports for the users they manage.	analytics and organization routers

Example: one chat question

- The user types a question on the AI insights page. The web app sends it to `POST /ai/chat`.
- The backend computes today's metrics for that user.
- It embeds the question and searches the user's own earlier daily summaries (previous 30 days, today excluded, top 5, similarity of at least 0.25).
- It builds one prompt from the question, today's metrics, the retrieved history and the recent conversation.
- The LLM replies, and the answer appears in the chat. If no provider responds, the user gets a short unavailable message.

4. Activity tracking in the extension

The extension is a Manifest V3 service worker with the permissions `tabs`, `storage`, `idle`, `notifications` and `alarms`. Each browser event leads to one or more backend calls.

BROWSER EVENT	WHAT background.js DOES	BACKEND CALL

Tab activated, page finished loading, or Chrome window focused	End the running activity.
	Start one for the new site.
	Record a tab switch if tab or site changed.
		`PUT /activity/end/{id}`
		`POST /activity/start`
		`POST /activity/tab-switch`

Tab closed	End the running activity if it belonged to that tab. Keep the site name so the next tab counts as a switch.	`PUT /activity/end/{id}`
Chrome reports idle or active (checked every 60 seconds)	Switch status between ACTIVE and IDLE.	`POST /activity/status`
Alarm every 5 minutes	Resend the current status as a heartbeat.	`POST /activity/status`
Alarm every 30 seconds	Fetch the latest unread notification, show it in Chrome, mark it read.	`GET /notifications/latest` `PATCH /notifications/{id}/read`
Alarm every minute	Ask whether the current site matches the planned session.	`POST /planner/check-focus`

Design choices that keep the data clean

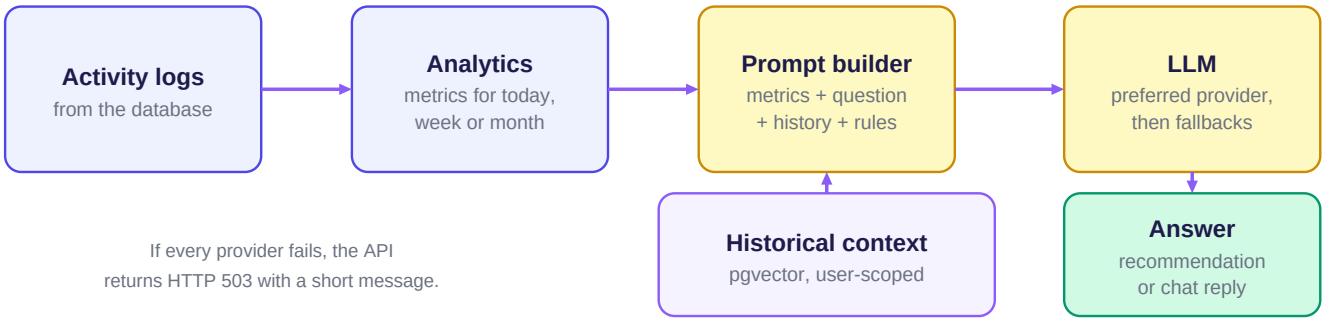
- Chrome stops an idle service worker after about 30 seconds, so the tracked tab, URL and website are kept in `chrome.storage` and not in variables.
- Events run through a queue, one at a time. Tab activation, page load and window focus often fire together, and handling them in parallel used to create duplicate activities.
- Pages such as `chrome://` and the new-tab page are not tracked. Visiting one stops the timer, so its time is not credited to the previous site.
- A tab switch is a change of tab or of website. Moving between pages of the same site in one tab does not count.
- Website names come from the hostname (for example github.com), because page titles change while a page loads and would create switches that never happened.

5. Analytics, focus score and recommendations

The dashboard and the AI layer read the same numbers. The analytics service computes them from activity and status logs for a chosen period.

Metric	Meaning
Active and idle time	Time the user was active or idle, from the status events the extension sends.
Browser time	Total time spent on tracked websites.
Productive and non-productive time	Browser time split by the productivity value of each website.
Focus score	Active time divided by active plus idle time, as a percentage. Browser time is not used as the denominator.
Tab switches	Number of recorded tab or website changes.
Top websites and categories	Where the time went, ranked.

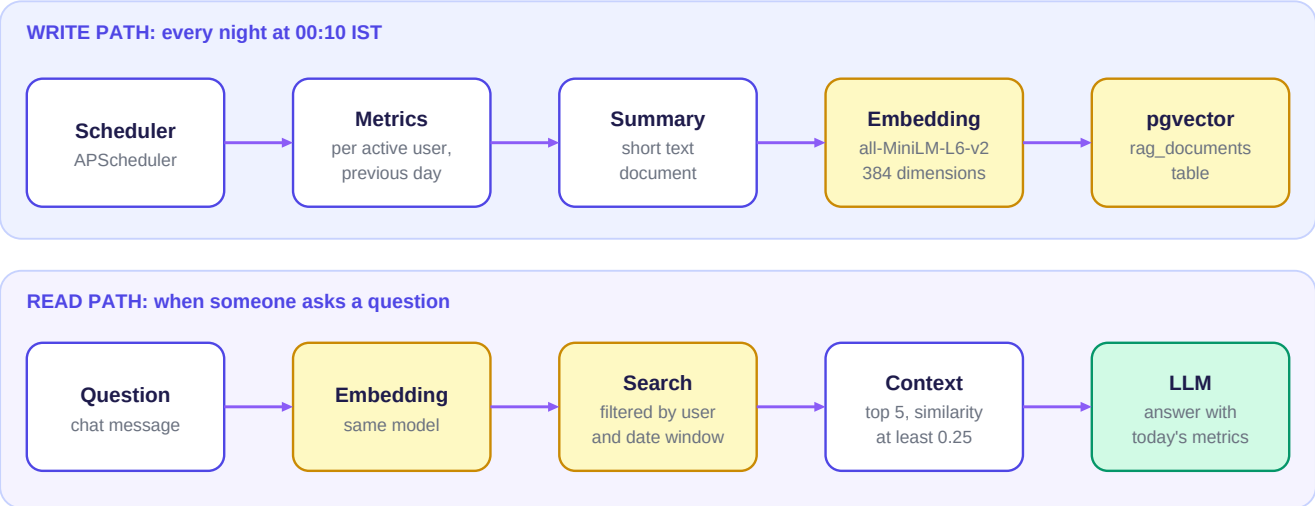
From numbers to advice



- Recommendations come in three periods: today, week and month. Each one is built from that period's metrics plus relevant history from earlier periods.
- If there is no activity at all today, the API returns a fixed message and does not ask the LLM to interpret zero data. Week and month still go through the normal flow.
- The system prompt tells the model to use only the supplied data, keep historical dates exact, and finish with complete sentences.
- Organization recommendations use the same flow with organization-level metrics.
- If every provider fails, the user sees a short message and the API returns HTTP 503.

6. Memory: the RAG pipeline

Yesterday's focus is as useful as today's, so recommendations and chat can look back. The write path runs once a night. The read path runs whenever someone asks.



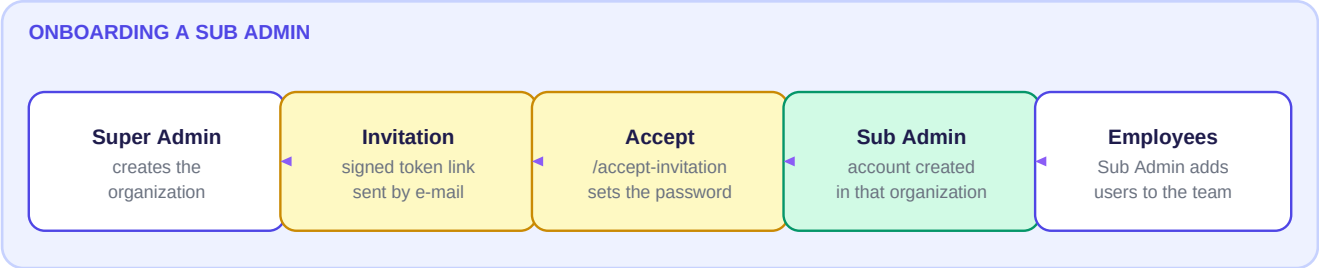
Retrieval rules

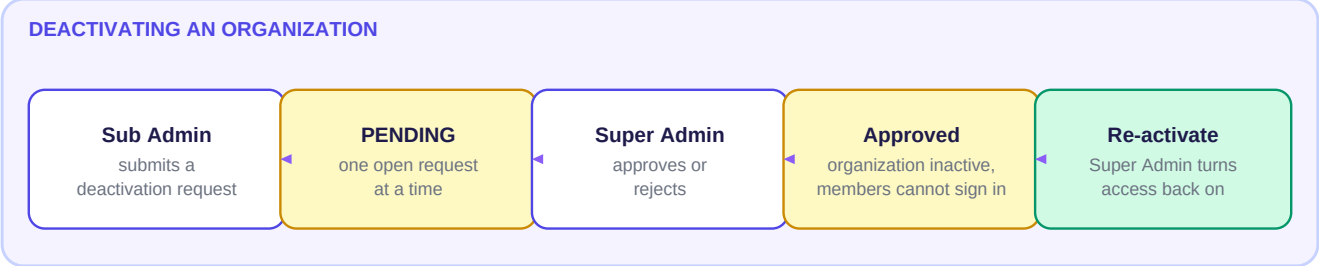
Setting	Value
Write schedule	Daily at 00:10, Asia/Kolkata time, through APScheduler inside the backend.
What is written	One daily summary per active user who has activity on the target date, with organization-level summaries as well. Users with no activity are skipped.
Embedding	all-MiniLM-L6-v2, normalized, 384 dimensions. The vector store rejects any other length.
Chat window	The previous 30 days, today excluded, up to 5 documents, minimum similarity 0.25.
Week recommendation	The 7-day period immediately before the rolling last 7 days.
Month recommendation	The 30-day period immediately before the rolling last 30 days.
Data isolation	User and organization filters are part of the SQL query, so another person's rows are never fetched and filtered out afterward.
Trust order	Today's metrics are authoritative. Retrieved history is supporting context and its dates are preserved exactly.

To fill the table for days that already exist, run `python backfill_rag.py` after `create_rag_table.py`.

7. Organizations, invitations and deactivation

Every account belongs to one of three roles, and organizations have an on and off state. These two flows are the ones that change who can sign in.





Rules worth knowing

- Only a Super Admin creates organizations and sends Sub Admin invitations. Invitation links are signed tokens, and the accept step refuses links that are expired, tampered with, already used, for the wrong role, or for a missing or inactive organization.
- A Sub Admin can have only one pending deactivation request at a time. A rejection leaves the organization running.
- Approval marks the organization inactive. Its members are then blocked at login, and invitations into it are refused until the Super Admin re-activates it.
- Approvals and rejections create notifications for the people involved.
- Public registration cannot choose a role or an organization. Any role or organization sent in the request is ignored.

Security and data isolation

- Passwords are hashed. Sessions use JWT access and refresh tokens, and role-based access control limits what each role can reach.
- Super Admin data, Sub Admin organization data and individual user data are kept apart, in the API and in the RAG queries.
- Secrets such as the secret key, database URL and API keys live in environment variables. `.env` is excluded from version control.

8. Focus planner and notifications

The planner is how FocusGuard checks intent against behavior. A user plans sessions by category with a start and end time. The extension then asks once a minute how it is going.

Planner status	When it appears
NO_PLAN	The user has no plan for today.
NO_SESSION	There is a plan, but the current time is outside every planned session.
NO_ACTIVITY	A session is running and no activity has been recorded yet.
ON_TRACK	The category of the latest activity matches the planned category.
OFF_TRACK	The categories differ. The message names both, for example planned Development but currently using Entertainment, and is translated to the user's language.

Notifications

Type	Triggered by
BREAK_REMINDER	A long run of continuous active time without a break.
NON_PRODUCTIVE_ACTIVITY	Non-productive time passing its threshold.
IDLE_ALERT	Idle time passing its threshold, followed by a welcome-back message.
Organization events	Deactivation approved or rejected, Sub Admin welcome, and similar account messages.

The backend stores each notification per user. The extension polls `/notifications/latest` every 30 seconds, shows the result as a Chrome notification that stays until dismissed, and marks it read. Each user sees only their own.

Before production: the break, idle and non-productive thresholds in `app/services/notification_rules.py` are currently set to 20 seconds each. That suits testing. Raise them for real use.

9. Data model and running the project

Table	Holds
organizations, deactivation_requests	Organizations and their deactivation requests with status.
users	Accounts: name, e-mail, hashed password, role, organization, preferred language, active flag.
activity_logs	One row per visit: URL, website, category, productivity, start, end and duration.
website_categories	Domain to category and productivity lookup, seeded at startup.
tab_switch_logs	From-website and to-website for each recorded switch.
user_status, user_status_logs	Current and historical ACTIVE or IDLE status.
notifications	Per-user notifications with type, text and read state.
focus_plans, focus_plan_items	A day's plan and its sessions (category, minutes, start, end).
translations, dynamic_translations	Cached interface and message translations.
rag_documents	Daily summaries with user, organization, date and a 384-dimension embedding.

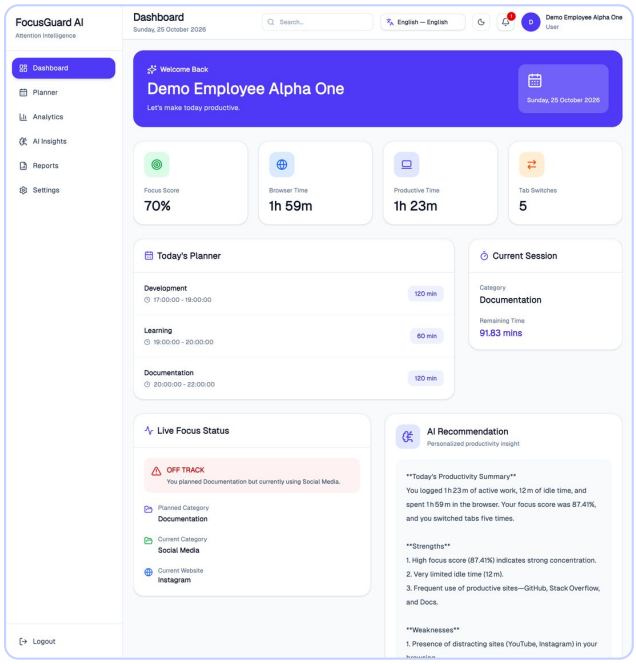
Running it locally

Part	Command	Address
Everything (macOS)	./focusguard.sh	starts PostgreSQL, backend and frontend
Backend	uvicorn app.main:app --reload --port 8000	http://127.0.0.1:8000
Frontend	cd focusguard-ui && npm run dev	http://localhost:5173
Extension	chrome://extensions, Developer mode, Load unpacked	folder chrome_extension
pgvector setup	enable_pgvector.py, check_pgvector.py, create_rag_table.py	run once
Demo data	python qa/seed_demo.py	needs the backend running

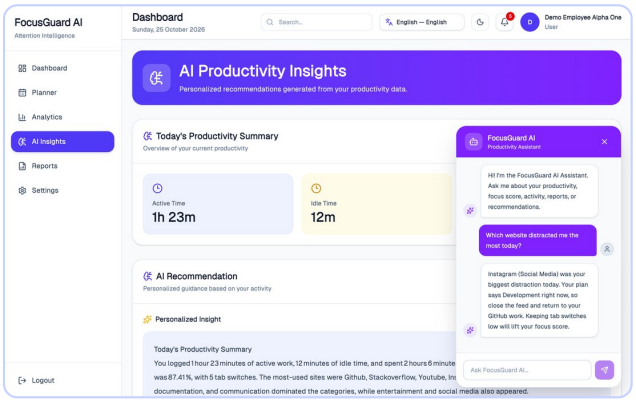
The extension talks to `127.0.0.1:8000` while `USE_LOCAL_BACKEND` in `chrome_extension/config.js` is true, and to the hosted backend otherwise. The code references a Vercel frontend and a Render backend for deployment. The dashboard sends each role to its own area: `/user`, `/subadmin` or `/superadmin`.

10. Screens

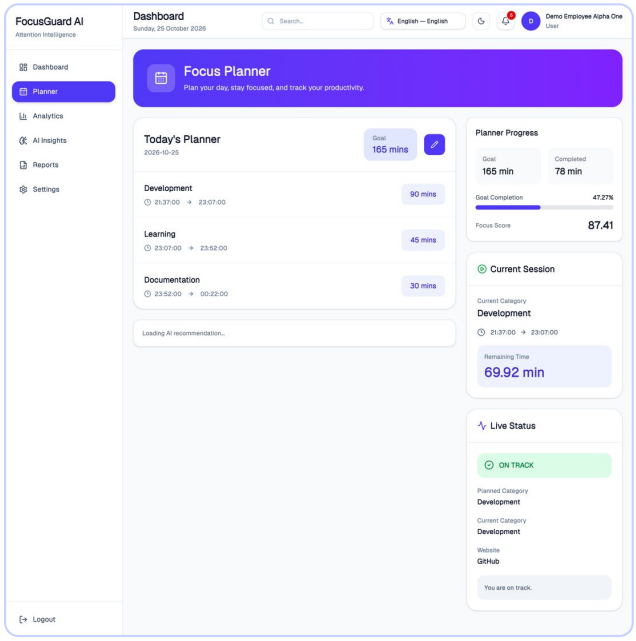
The same workflow as users see it.



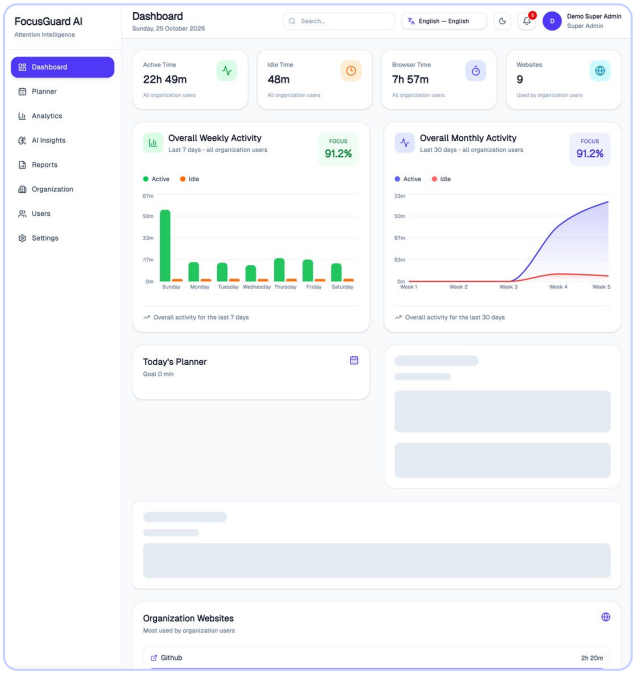
Employee dashboard



AI insights and chat assistant



Focus planner with a live session



Super Admin dashboard